Primer: Adversarial Inverse Reinforcement Learning (AIRL)

econirl package documentation

1 Problem and Motivation

Standard IRL methods recover a reward function from observed behavior, but the recovered reward is not robust. If the transition dynamics change (new environment, counterfactual policy), a reward estimated under the original dynamics may produce incorrect predictions. The problem is that IRL rewards are identified only up to a potential-based shaping term. For any function $h : \mathcal{S} \to \mathbb{R}$, the shaped reward

$$r'(s, a) = r(s, a) + \gamma h(s') - h(s)$$

induces the same optimal policy as $r(s, a)$ under the original dynamics. But under different dynamics, the shaping term $\gamma h(s') - h(s)$ evaluates differently, and the shaped reward no longer rationalizes the same behavior.

? propose AIRL to solve this. The key idea is to decompose the discriminator into a reward term $g_\theta(s, a)$ and a shaping potential $h_\phi(s)$, then learn them jointly so that g_θ converges to the true reward (not a shaped version). **?** extend AIRL with anchor constraints and latent heterogeneity for structural estimation of dynamic discrete choice models. We call this extension Anchored AIRL (AAIRL).

The core contribution relative to other IRL methods is *disentanglement*: AIRL separates the transferable reward from the non-transferable value function within the adversarial training loop, rather than recovering an entangled quantity and hoping it transfers.

2 Model

2.1 Notation

2.2 Objective

The agent solves the maximum causal entropy problem

$$\max_{\pi} \; \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t \big(r(s_t, a_t) + \sigma \mathcal{H}(\pi(\cdot \mid s_t)) \big) \right] \tag{1}$$

where $\mathcal{H}(\pi(\cdot \mid s)) = -\sum_a \pi(a \mid s) \log \pi(a \mid s)$ is the Shannon entropy. The optimal policy satisfies the softmax relationship

$$\pi^*(a \mid s) = \frac{\exp(Q^*(s, a)/\sigma)}{\sum_{a'} \exp(Q^*(s, a')/\sigma)} = \frac{\exp(A^*(s, a)/\sigma)}{\sum_{a'} \exp(A^*(s, a')/\sigma)}. \tag{2}$$

Taking the log of both sides gives

$$\log \pi^*(a \mid s) = \frac{1}{\sigma} \big(Q^*(s, a) - V^*(s) \big) = \frac{1}{\sigma} A^*(s, a). \tag{3}$$

Symbol	Meaning
$s \in \mathcal{S}$	State (finite or continuous)
$a \in \mathcal{A}$	Action (finite: $\lvert\mathcal{A}\rvert = k$)
s'	Next state after taking action a in state s
$\gamma \in [0, 1)$	Discount factor
$\sigma > 0$	Scale parameter (entropy regularization; $= 1$ in IRL convention)
$r^*(s, a)$	True (unknown) reward function
$\pi^*(a \mid s)$	Expert policy (observed through demonstrations)
$\tau = (s_0, a_0, s_1, a_1, \ldots)$	Trajectory
$P(s' \mid s, a)$	Transition kernel (may be known or unknown)
$V^*(s)$	True optimal value function
$Q^*(s, a)$	True optimal action-value function
$A^*(s, a) = Q^*(s, a) - V^*(s)$	Advantage function
$g_\theta(s, a)$	Learned reward approximator (parameterized by θ)
$h_\phi(s)$	Learned shaping potential $\approx V(s)$ (parameterized by ϕ)
$\pi_\omega(a \mid s)$	Learned policy (parameterized by ω)
$f_{\theta,\phi}(s, a, s')$	Discriminator logit
$D_{\theta,\phi}(s, a, s')$	Discriminator output $\in [0, 1]$

Table 1: Notation for AIRL.

2.3 Discriminator Structure

AIRL uses a structured discriminator that decomposes into reward and shaping components:

$$f_{\theta,\phi}(s, a, s') = g_\theta(s, a) + \gamma\, h_\phi(s') - h_\phi(s), \tag{4}$$

$$D_{\theta,\phi}(s, a, s') = \frac{\exp\big(f_{\theta,\phi}(s, a, s')\big)}{\exp\big(f_{\theta,\phi}(s, a, s')\big) + \pi_\omega(a \mid s)}. \tag{5}$$

The discriminator classifies transitions as "expert" (from data) or "generated" (from the current policy). The key structural assumption is in Equation (**??**): the logit f is a sum of a state-action reward term g_θ and a potential-based shaping term $\gamma h_\phi(s') - h_\phi(s)$. This mirrors the Bellman decomposition $Q(s, a) = r(s, a) + \gamma\mathbb{E}[V(s')] - V(s) + V(s)$.

3 Identification

3.1 The Reward Shaping Ambiguity

IRL rewards are identified only up to potential-based shaping (**?**). For a single agent in a single environment, the observed policy constrains $\lvert\mathcal{S}\rvert \cdot (\lvert\mathcal{A}\rvert - 1)$ equations but the reward has $\lvert\mathcal{S}\rvert \cdot \lvert\mathcal{A}\rvert$ unknowns, leaving $\lvert\mathcal{S}\rvert$ free variables that correspond to the value function (**?**).

3.2 AIRL Disentanglement (Fu et al. 2018, Theorem 1)

? show that at the optimum of the adversarial objective, if: (a) the state-action space is finite, (b) the reward is state-only (g_θ depends on s but not a), and (c) the MDP satisfies a "decomposability" condition (all states are linked), then the recovered reward g_θ equals $r^*(s) + c$ for some constant c.

The decomposability condition (Definition B.1 in **?**) requires that all states are reachable from all other states, which rules out absorbing states. This limits the applicability to DDC models where exit leads to an absorbing state.

3.3 Anchor Identification (Lee, Sudhir, Wang 2026)

? extend the result to *action-dependent* rewards using anchor constraints. If one action (exit) has known zero reward and one state (absorbing) has known zero value:

$$g_\theta(s, a_{\text{exit}}, z) = 0 \;\; \forall s, z, \qquad h_\phi(s_{\text{abs}}, z) = 0 \;\; \forall z, \tag{6}$$

then the decomposition $f = g_\theta + \gamma h_\phi(s') - h_\phi(s)$ is unique at convergence. The anchors pin down the $|\mathcal{S}|$ free variables: the zero-value anchor eliminates the constant, and the zero-reward anchor on the exit action provides the reference utility normalization standard in discrete choice models (cf. **?**).

This means $g_\theta(s, a, z) = r^*(s, a, z)$ and $h_\phi(s, z) = V^*(s, z)$ at the optimum, enabling welfare-valid counterfactual analysis.

4 Estimation Algorithm

Algorithm 1 AIRL with Anchor Constraints

Require: Expert transitions $\mathcal{D}_{\text{obs}} = \{(s, a, s')_n\}_{n=1}^N$, discount γ, learning rate α, anchor action a_{exit}, absorbing state s_{abs}

Ensure: Reward parameters θ, shaping parameters ϕ, policy parameters ω

Initialize $\theta_0, \phi_0, \omega_0$; set $\pi_\omega \leftarrow$ uniform
for round $= 1, 2, \ldots$ until convergence **do**
  // Step 1: Generate policy transitions
  Sample M transitions $\mathcal{D}_{\text{sim}} = \{(s, a, s')_m\}$ by rolling out π_ω
  // Step 2: Update discriminator (reward + shaping)
  for $j = 1, \ldots, J$ (discriminator steps) **do**
    Compute logits: $f_n = g_\theta(s_n, a_n) + \gamma\, h_\phi(s'_n) - h_\phi(s_n)$
    Compute discriminator: $D_n = \exp(f_n)/(\exp(f_n) + \pi_\omega(a_n \mid s_n))$
    $_{\text{disc}} = -\frac{1}{N}\sum_{\mathcal{D}_{\text{obs}}} \log D_n - \frac{1}{M}\sum_{\mathcal{D}_{\text{sim}}} \log(1 - D_m)$
    $\theta, \phi \leftarrow \theta, \phi - \alpha\, \nabla_{\theta,\phi}\, \text{disc}$
    Enforce anchors: project $g_\theta(s, a_{\text{exit}}) = 0\ \forall s$; $h_\phi(s_{\text{abs}}) = 0$
  end for
  // Step 3: Update policy
  Compute reward signal: $\tilde{r}(s, a, s') = f_{\theta,\phi}(s, a, s') - \log \pi_\omega(a \mid s)$
  Update ω via policy optimization on $\max_{\pi_\omega} \mathbb{E}[\sum_t \gamma^t \tilde{r}]$
  // Step 4: Check convergence
  if $\max_{s,a} |\pi_\omega^{\text{new}}(a \mid s) - \pi_\omega^{\text{old}}(a \mid s)| < \varepsilon$ **then**
    break
  end if
end for
return g_θ (recovered reward), h_ϕ (recovered value function), π_ω (recovered policy)

Generator options. Step 3 admits two implementations depending on the state space.

Setting	Generator	Implementation
Tabular (finite $\mathcal{S}$, known P)	Soft value iteration	Exact Bellman solve with conservative mixing ($\alpha_{\text{policy}} < 1$)
High-dimensional (continuous $\mathcal{S}$)	PPO / policy gradient	Neural policy network, clipped surrogate objective

In the tabular case, exact value iteration finds the optimal policy in one step, which causes the discriminator to overreact and rewards to diverge. Conservative policy iteration mixes old and new policies: $\pi^{\text{new}} = (1 - \alpha_{\text{policy}})\,\pi^{\text{old}} + \alpha_{\text{policy}}\,\pi^{\text{VI}}$. This dampens the update and allows the discriminator to converge to $D = 1/2$ gradually.

At convergence. When $D = 1/2$ everywhere, the discriminator cannot distinguish expert from generated transitions. The logit satisfies $f_{\theta,\phi}(s, a, s') = \log \pi^*(a \mid s) = A^*(s, a)$. With the anchor constraints enforced, the decomposition is unique:

$$g_\theta(s, a) = r^*(s, a), \qquad h_\phi(s) = V^*(s).$$

5 Counterfactual Analysis

The structural reward recovered by AIRL enables counterfactual policy evaluation under modified environments. **?** define three types.

Type A: Within-environment extrapolation. Change a policy parameter (e.g., introduce wait-for-free access) while keeping the transition dynamics fixed. Re-solve the Bellman equation with the recovered reward under the new action set. This is the simplest counterfactual because only the choice set changes, not the dynamics.

Type B: Segment-specific policies. Apply different policy parameters to different latent segments (e.g., longer wait-times for price-insensitive users, shorter for patient users). Requires heterogeneous estimation to distinguish segments, then per-segment policy evaluation.

Type C: Content-dependent policies. Exploit variation in state features (e.g., episode quality) to set different pricing rules for different states. This requires the reward to depend on content features, which AIRL can recover through the discriminator's dependence on state.

All three types use the same procedure: take the recovered g_θ, modify the transitions or feature matrix to encode the counterfactual scenario, re-solve for the optimal policy using value iteration or PPO, and simulate behavior under the new policy.

6 Connection to econirl Code

The AIRL estimator in econirl lives at `src/econirl/estimation/adversarial/airl.py`. The heterogeneous extension with EM over latent segments is at `airl_het.py`.

Paper concept	Code class / function	Config parameter
$g_\theta(s, a)$ (reward)	`reward_params` (linear) or `reward_matrix` (tabular)	`reward_type="linear"`
$h_\phi(s)$ (shaping)	`V` (value function from VI)	`use_shaping=True`
$\pi_\omega(a \mid s)$ (policy)	`policy` from `SoftBellmanOperator`	`generator_solver="hybrid"`
Anchor constraints	Enforced after each discriminator step	`exit_action`, `absorbing_state`
Conservative VI	Old/new policy mixing	`policy_step_size=0.1`
Weight decay on θ	AdamW optimizer	`reward_weight_decay=0.01`

Table 2: Mapping from paper to code.

Example usage.

```
from econirl.estimation.adversarial.airl import AIRLEstimator, AIRLConfig

config = AIRLConfig(
    reward_type="linear",
    max_rounds=500,
    reward_lr=0.01,
    policy_step_size=0.1,
    use_shaping=True,
    verbose=True,
)
result = AIRLEstimator(config).estimate(panel, utility, problem, transitions)
```

The serialized content example at `examples/serialized-content/lsw_replication.py` demonstrates AIRL on a 51-state, 3-action wait-for-free platform with known ground truth.

7 Comparison to Other Estimators

	NFXP	MCE-IRL	AIRL	GLADIUS
Requires known $P(s' \mid s, a)$	Yes	Yes	No (PPO) / Yes (VI)	Yes
Reward parameterization	Linear	Linear	Neural or linear	Neural
Handles continuous states	No	No	Yes (PPO)	Yes
Action-dependent rewards	Yes	Yes	With anchors	Yes
Identification strategy	Parametric MLE	Feature matching	Disentanglement + anchors	Bellman decomposition
Inference (standard errors)	Yes	Yes	Bootstrap	Bootstrap

Table 3: AIRL in context.

AIRL's advantage over NFXP and MCE-IRL is that it can handle continuous state spaces when using PPO as the generator. Its advantage over GLADIUS is the disentanglement guarantee (Theorem 1 in **?**): the recovered reward is structurally meaningful, not just a sufficient statistic for the policy. GLADIUS recovers Q-values that are identified only up to a state-dependent constant in the IRL setting.

AIRL's disadvantage on tabular problems is computational: the adversarial training loop is slower and less stable than direct MLE (NFXP) or feature matching (MCE-IRL). On a 51-state serialized content environment, NFXP recovers all 6 reward parameters within 15 percent of the true values in 13 seconds. AIRL with conservative policy iteration requires 500 rounds and careful tuning of the learning rate, weight decay, and policy step size.

References.

Fu et al. (2018) "Learning Robust Rewards with Adversarial Inverse Reinforcement Learning." ICLR.

Lee, Sudhir, Wang (2026) "Modeling Serialized Content Consumption: Adversarial IRL for Dynamic Discrete Choice."

Ng et al. (1999) "Policy Invariance Under Reward Transformations." ICML.

Cao, Cohen (2021) "Identifiability in Inverse Reinforcement Learning." NeurIPS.

Kim et al. (2021) "Reward Identification in Inverse Reinforcement Learning." ICML.

Magnac, Thesmar (2002) "Identifying Dynamic Discrete Decision Processes." Econometrica.

Kang, Yoganarasimhan, Jain (2025) "GLADIUS: ERM for Offline IRL."